Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Code Shield : Source Code Similarity and Plagiarism Detection System

Team Name: PlagX	*PlagX* *DSCPP-III-2026-T353*
Team member 1 (Team Lead) *DUBEY, GAURAV–* *2510370051* *2510370051@geu.ac.in*	*DUBEY, GAURAV–* *2510370051* *2510370051@geu.ac.in* 
Team member 2 *JANDIAL, HARSHIT–* *2510350032* *2510350032@geu.ac.in*	*JANDIAL, HARSHIT–* *2510350032* *2510350032@geu.ac.in* 

Project Abstract

Code Shield is our C++ project for detecting copied code. Anyone who has marked programming assignments knows how long it takes to compare two programs by eye, and students make it harder by renaming a few variables, adding comments or changing the spacing. The code looks different even though nothing important has changed.

We did not want the tool to be fooled by this, so it does not read files as plain text. Comments are removed first, then every variable is given the same name. What remains is split into tokens, the tokens are grouped into k-grams, and each k-gram is hashed. Winnowing selects which hashes to keep as fingerprints, and two files are compared through those fingerprints to give a similarity percentage.

C, C++ and Python files are currently supported. The output shows the percentage, the token, k-gram and fingerprint counts, and a label of high, moderate or low. A high number should not be read as a verdict, because it cannot prove that anyone copied. It only suggests that the two programs deserve a closer look by a person. Our aim was a tool that is simple and saves time.

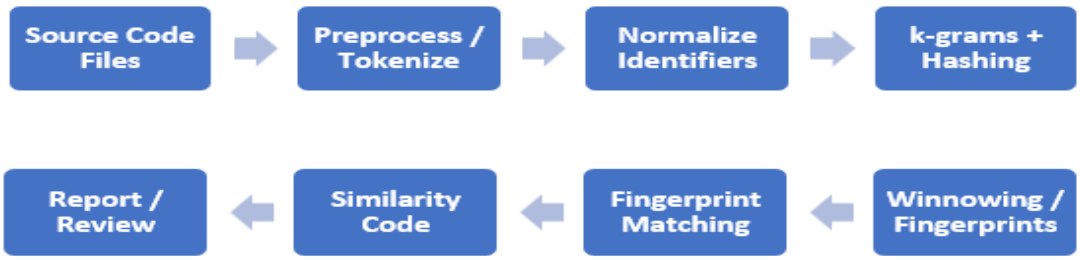
Updated Project Approach and Architecture

This time we focused on how the code is built rather than how its text looks. The program is written in C++ and split into classes, each of which does one thing only.

Using it is simple. You enter the paths of two files, and the extensions are checked; anything other than C, C++ or Python is rejected. After the files load, the CodePreprocessor strips out the comments and the Tokenizer makes tokens. Variable names turn into one shared identifier, keywords stay as they are, and every string or number becomes a standard token.

The KGramGenerator then takes five consecutive tokens to form each k-gram, and the HashGenerator works out a 64-bit FNV-1a hash for each one. Winnowing with a window of four selects the fingerprints. Finally, the SimilarityChecker compares the fingerprints of the two files using Jaccard similarity, and the percentage it gets is what the user sees.

Nothing outside standard C++ is used. The headers are iostream, fstream, string, vector, unordered_set, algorithm, cctype, cstdint, iomanip, limits and sstream. It runs on a single machine from the command line, so no network or communication protocol is needed. The user sees file details, token counts, fingerprint counts and the final result.



Tasks Completed

Task Completed	Team Member
Started the project and wrote the file-reading code.	Gaurav Dubey
Wrote the tokenizer for C, C++ and Python.	Gaurav Dubey
Coded comment removal and the preprocessing step.	Gaurav Dubey
Implemented identifier normalization, leaving keywords untouched.	Gaurav Dubey
Wrote the code that builds k-grams from five tokens.	Harshit Jandial
Added 64-bit FNV-1a hashing for each k-gram.	Harshit Jandial
Coded Winnowing so that only the key fingerprints are kept.	Harshit Jandial
Set up how fingerprints are selected inside the Winnowing step.	Harshit Jandial
Worked out the similarity percentage and the result label.	Harshit Jandial
Connected all modules into one command-line program that takes two source files and prints the comparison.	Gaurav Dubey & Harshit Jandial

Challenges/Roadblocks

Our first problem was the score itself, since comments, coding style and variable names all changed it. Removing comments and normalizing identifiers early in the pipeline fixed most of this.

Then came the k-gram and window sizes. We picked 5 and 4, which work, but we have not tested them enough to be sure they are the best choice. Very short programs are also a problem, as they give too few fingerprints to compare.

The tool has limits too. It takes only two files at a time and cannot show which lines matched. Common programming patterns can also raise the score even when nothing was copied.

Next, we plan to try more code samples and tune the settings. We also want multiple-file comparison and stored fingerprint positions so that matching sections can be shown. Code given by the instructor should be handled separately as well, so the results are more accurate.

Project Outcome/Deliverables

We have built a working C++ application that finds similarities between source code files. It accepts C, C++ and Python files and compares two programs at a time.
Internally, the steps are: remove comments, normalize variable names, tokenize, build k-grams, hash them, and keep fingerprints with Winnowing. The percentage comes from Jaccard similarity and is then labeled high, moderate or low.
The final product runs on the command line. It shows file details, token counts, fingerprint counts and the similarity. Teachers and students can both use it during code review. The score is only a pointer, and a person has to read the code before anyone says it was plagiarized.

Progress Overview

The main development is finished. The program reads C, C++ and Python files, removes comments, normalizes identifiers, makes tokens and k-grams, hashes them, applies Winnowing and gives the similarity of two files.
Everything works from the command line. Three planned features are not done yet: comparing many files at once, showing exact matching sections, and a detailed report. We are a little behind on these, and they are the first things for the next stage.
The project is going well and the main algorithms work. We are now testing with more code samples and want the results to be more accurate.

Codebase Information

Repository link: harshitjandialaj321-cell/Code-Shield: Code Shield: Source Code Similarity and Plagiarism Detection System

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Missing file input	Pass	Error shown when the file is missing.
Identical source code files	Pass	Result is 100% similarity.
Different spacing and indentation	Pass	Extra spaces and tabs make no difference.
Comment removal	Pass	Comments are removed before tokenizing.
Identifier normalization	Pass	Every variable name becomes one identifier.
Two different programs	Pass	The score is low, as expected.
Empty source files	Pass	Empty files are handled without a crash.
Unsupported file extensions	Pass	Error shown for unsupported extensions.

Deliverables Progress

The main Code Shield application is finished and supports C, C++ and Python files. Comment removal, identifier normalization, tokenization, k-grams, hashing and Winnowing are implemented, and the program shows the similarity percentage for two source files.
Some features are still pending: comparing multiple files, showing the exact matching sections, a detailed similarity report, and handling the instructor's code. We are still testing and working on accuracy.
The core deliverables are complete, and the rest will be built in the next stage.